MentorBot — Detailed Architecture Map

Below is the **detailed functional architecture** of MentorBot, showing the flow from **user input** → **analysis** → **adaptation** → **LLM generation** → **evaluation**.

User Interaction Layer

User
↓
UI Interface (Streamlit / Gradio)
↓
Input: Question / Answer / Feedback

Handles user input/output (chat UI, answer box, feedback graph).

- File: `ui/app.py`
- Role: Capture learner text, display adaptive responses, and visualize progress.

Understanding Analyzer (Cognitive Assessment)

User Input ➞ Text Complexity Analyzer ➞ Level Detector ➞ Domain Classifier

Modules & Models:

Component	Purpose	Model Used	Output
`text_complexity.py`	Measures vocabulary depth, sentence difficulty	`textstat`	Flesch-Kincaid score, POS depth
`level_detector.py`	Detects user proficiency (Beginner → Advanced)	`facebook/bart-large-mnli` or `flan-t5-base`	Learner level
`domain_classifier.py`	Detects subject area (Math, Science, etc.)	`flan-t5-base`	Subject category

These steps help the system **understand the learner**, not just the question.

Adaptive Profile Manager (Personalization Engine)

Learner Level + Past Accuracy + Preferences
↓
Update → JSON Memory (user_profiles/user_X.json)

Module: `user_profile_manager.py`

Key Functions:

- Tracks learner level and topic familiarity.
- Updates scores after each question.
- Stores history to dynamically adjust difficulty.

This is what makes MentorBot *adaptive*, not static.

Context Builder (Knowledge Retrieval Layer)

Learner Query + Domain → Vector Embeddings → FAISS/ChromaDB Search → Context Chunk

Modules & Models:

- `context_retriever.py` — retrieves the top k relevant examples.
- Embedding Model: `sentence-transformers/all-MiniLM-L6-v2` or `intfloat/e5-base-v2`
- Backends: FAISS or Chroma for similarity search.

Purpose:
Provides factual grounding and examples related to the learner's topic, reducing hallucinations and improving relevance.

Adaptive Prompt Builder (Cognitive Prompting)

Input: Learner Level + Context + Query
Output: Custom Prompt Template

Example Template:

Explain this concept like you're teaching a {learner_level} student.
Topic: {domain}
Learner asked: "{question}"
Relevant examples: {retrieved_context}

Module: `adaptive_prompt_builder.py`
This ensures the **same base model** can teach at multiple levels by modifying how it's prompted — not by retraining.

Base LLM (Tutor Brain)

Adaptive Prompt → Base LLM → Response

Models:

- `mistralai/Mistral-7B-Instruct-v0.3`
- `microsoft/Phi-3-mini-4k-instruct`
- `meta-llama/Meta-Llama-3-8B-Instruct`

Module: `model_loader.py`
Handles loading and inference via `transformers` or `vLLM`.

Purpose:
Generates personalized responses, examples, and explanations depending on the prompt context and learner profile.

Evaluation & Feedback Loop (Learning Analytics)

User Response → Evaluator → Feedback → Profile Update

Modules:

Component	Purpose	Description
`evaluator.py`	Measures correctness, clarity, engagement	Compares learner answer vs reference
`feedback_generator.py`	Generates detailed text feedback	Uses LLM for reflective guidance
`progression_tracker.py`	Updates learner progress stats	Adjusts profile level dynamically

Data & Experiment Layer (Offline Analytics)

/data/ → Datasets, knowledge bases
/experiments/ → Run evaluations & ablation studies
/reports/ → Architecture diagrams, results, slides

Used mainly for *evaluation*, *benchmarking*, or *future fine-tuning*, not for real-time operations.

Complete MentorBot Architecture Diagram (Textual)

```
+-----------------------------------------------------------------------------------+
|                                 MENTORBOT (Dynamic)                               |
+-----------------------------------------------------------------------------------+
|                                                                                   |
| 1 User Input → Learner's question or response                                    |
|                                                                                   |
| 2 Understanding Analyzer                                                          |
|    ├─ NLP models compute:                                                         |
|    │   • Vocabulary complexity (textstat, POS depth)                              |
|    │   • Domain classification (math, science, etc.)                              |
|    │   • Question reasoning type (factual vs conceptual)                          |
|    ├─ Output: Learner Level ("Beginner", "Intermediate", "Advanced")              |
|                                                                                   |
| 3 Adaptive Profile Manager (JSON Memory)                                         |
|    ├─ Maintains user's current difficulty, accuracy, and preferences              |
|    ├─ Updates trend after each interaction                                        |
|                                                                                   |
| 4 Context Builder (LangChain / LlamaIndex)                                       |
|    ├─ Retrieves relevant examples from Knowledge Base                             |
|    ├─ Builds adaptive prompt:                                                     |
|    │     "Explain this like you're teaching a {level} student."                  |
|                                                                                   |
| 5 Base LLM (Hugging Face Model)                                                  |
|    ├─ Model: Mistral / Phi-3 / Llama-3                                            |
|    ├─ Generates personalized explanation / feedback                               |
|                                                                                   |
| 6 Evaluation & Feedback Loop                                                     |
|    ├─ Evaluates correctness, clarity, and engagement                              |
|    ├─ Updates learner profile and level dynamically                               |
|                                                                                   |
+-----------------------------------------------------------------------------------+
```

Why Use *Models* Instead of *Datasets* Here

MentorBot does not rely on datasets, but rather on models, since they offer intelligence, adaptability and real-time understanding that cannot be offered by the static datasets. Mistral, Phi-3, or Llama-3 are models that can reason, classify and generate text on-the-fly, which can enable MentorBot to work in inference mode - it dynamically responds without requiring further training data. Data sets on the other hand only have raw examples or facts and are not adaptive.

Models allow adaptivity that is, one can make changes to explanations or question difficulty in real-time according to the performance of each learner. On the contrary, datasets are fixed and cannot change with the user. Since MentorBot is to be used as a personalized tutor, the bot needs to be able to provide an instant response, change tone, style, and complexity in the moment, only language models can be used effectively.

Also, models can be personalized using prompt engineering and profiling of learners, which provides the ability to respond to the level and rate of learning peculiarities of a particular student. Datasets can only deliver pre-determined content, and they are not flexible. Knowledge generalization is also available through models since they are pre trained on large and diverse areas and can thus easily explain a variety of subjects such as math, science, or English. Lastly, regarding scalability, as soon as a model is incorporated, it is able to serve an unlimited number of learners at the same time, whereas datasets become bulky in size and slow to retrieve.

Simply, datasets are a source of reference material, yet models are the brains of the engine that converts existing non-adaptive information into intelligent, dynamic tutoring, and, therefore, they are essential to the dynamic learning process that MentorBot will be built on.

In short:

Datasets = raw materials.
Models = intelligent tutors built from those materials.
MentorBot combines both — datasets power *retrieval*, but models handle *understanding, generation, and adaptation*.